

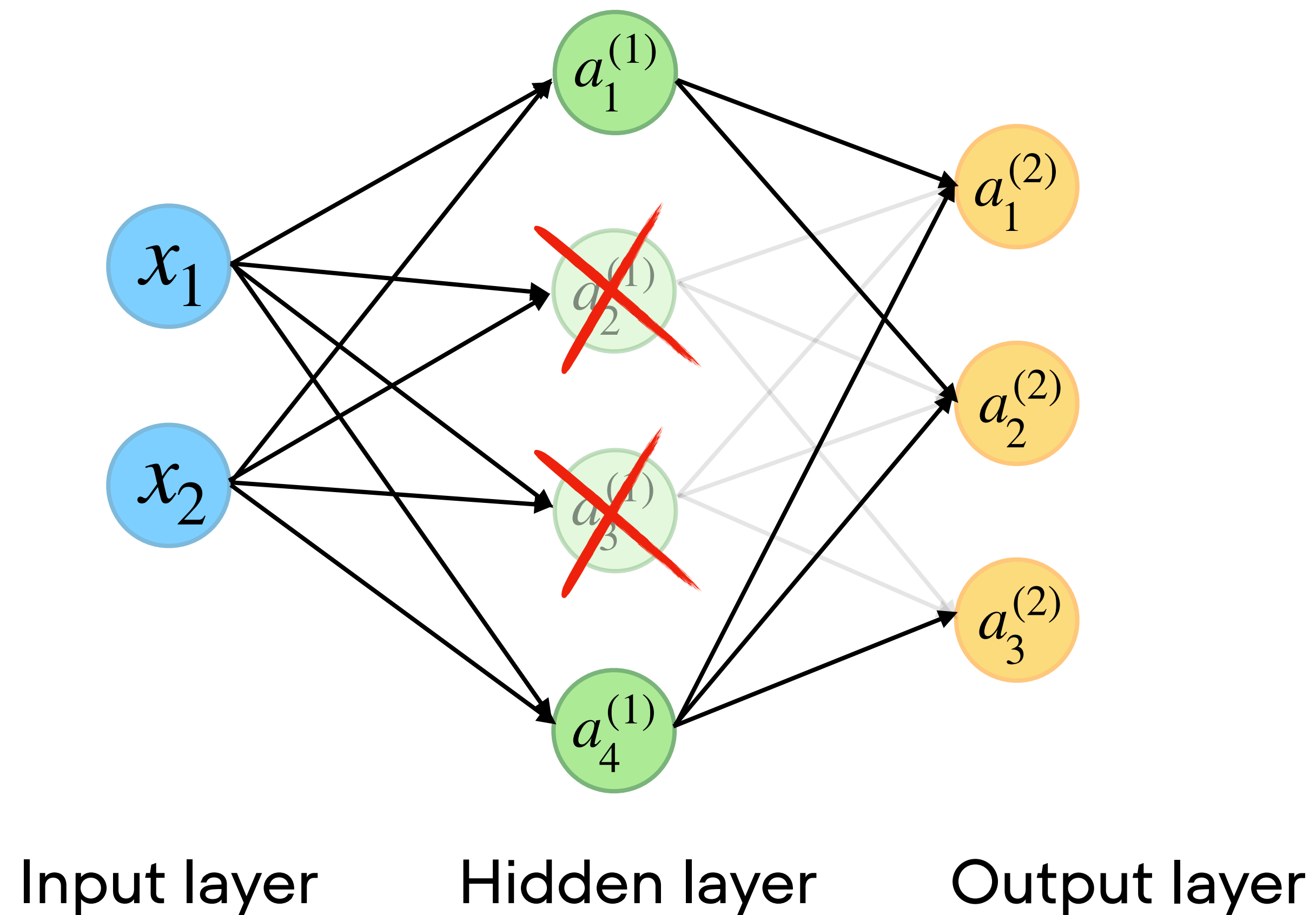
6.7

Reducing Overfitting With Dropout

Part 3: Adding Dropout Layers in PyTorch

Sebastian Raschka and the Lightning AI Team

Dropout in a nutshell: randomly drop nodes during training



And during inference?

And **during inference?**

Deterministic behavior!
Don't drop nodes!

Dropout in PyTorch

Dropout in PyTorch

```
class PyTorchMLP(torch.nn.Module):
    def __init__(self, num_features, num_classes):
        super().__init__()

        self.all_layers = torch.nn.Sequential(
            # 1st hidden layer
            torch.nn.Linear(num_features, 100),
            torch.nn.ReLU(),
            → torch.nn.Dropout(0.2),

            # 2nd hidden layer
            torch.nn.Linear(100, 50),
            torch.nn.ReLU(),
            → torch.nn.Dropout(0.5),

            # output layer
            torch.nn.Linear(50, num_classes),
        )

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        logits = self.all_layers(x)
        return logits
```

Dropout in PyTorch

```
class PyTorchMLP(torch.nn.Module):
    def __init__(self, num_features, num_classes):
        super().__init__()

        self.all_layers = torch.nn.Sequential(
            # 1st hidden layer
            torch.nn.Linear(num_features, 100),
            torch.nn.ReLU(),
            → torch.nn.Dropout(0.2),

            # 2nd hidden layer
            torch.nn.Linear(100, 50),
            torch.nn.ReLU(),
            → torch.nn.Dropout(0.5),

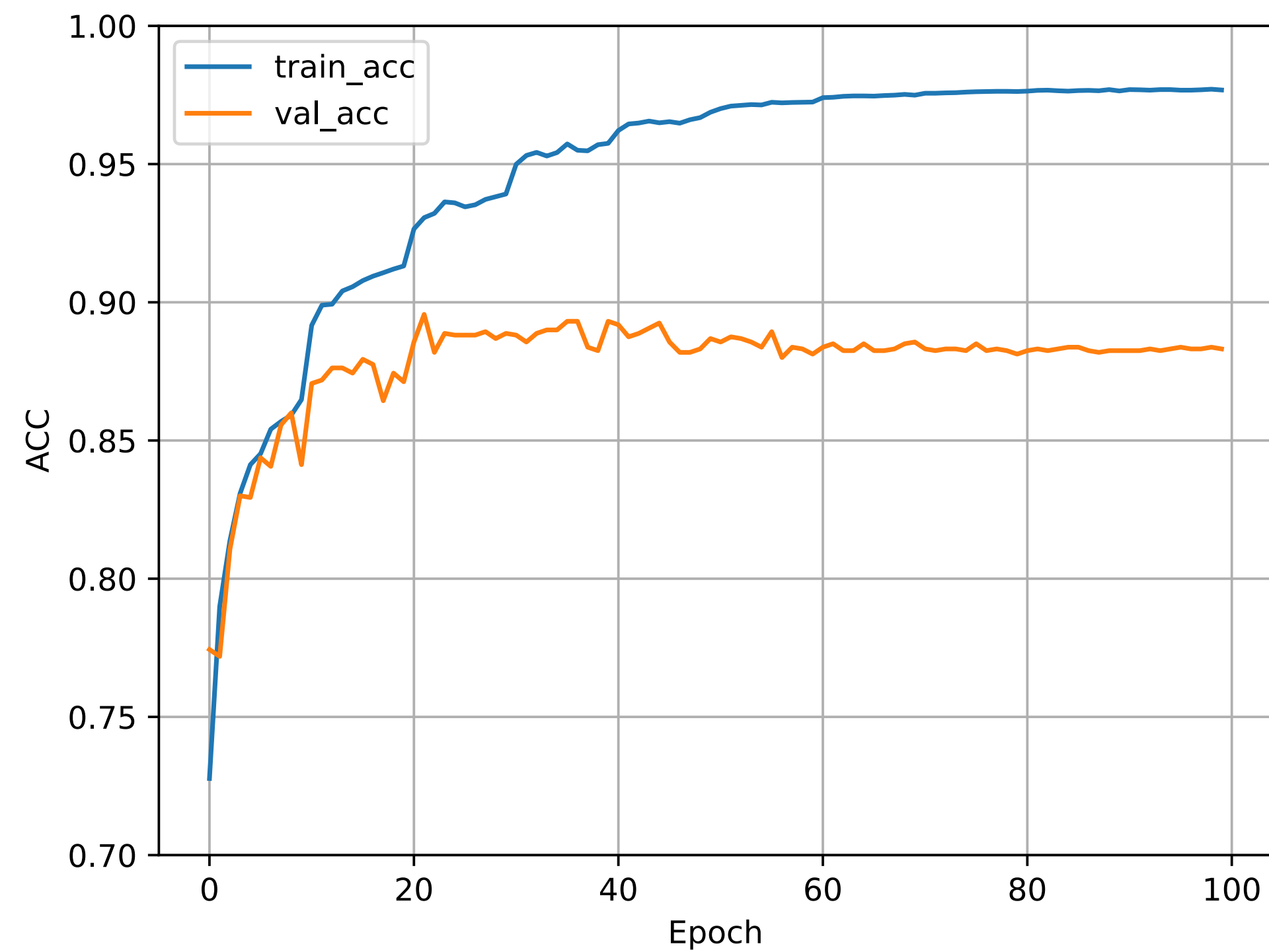
            # output layer
            torch.nn.Linear(50, num_classes),
        )

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)
        logits = self.all_layers(x)
        return logits
```

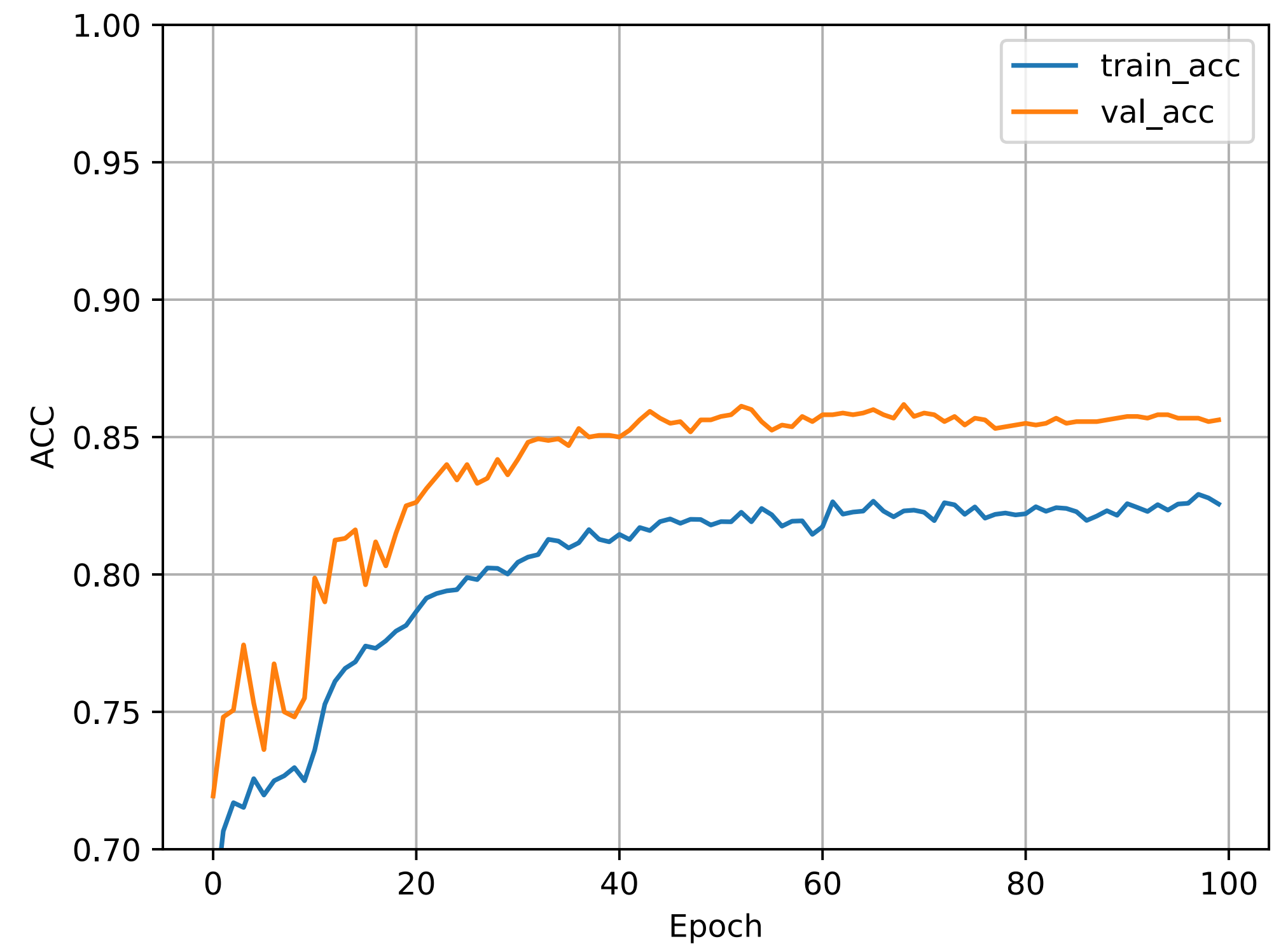
deterministic behavior when

`model.eval()`

Without Dropout



With Dropout



NEXT: Debugging Neural Networks